

ORM

Sequelize, *SQL through JS.*

Models, queries, associations, eager loading, and the sharp edges of hiding SQL behind JavaScript.

models

CRUD

associations

include

Op

THE IDEA

An ORM maps rows to objects.

Raw SQL	Sequelize
SELECT * FROM users WHERE id = 3	User.findByPk(3)
INSERT INTO users ...	User.create({ name: "Ada" })
DELETE FROM users WHERE id = 5	user.destroy()

same database. different interface.

WHY AND WHY NOT

Sequelize removes boilerplate, not thinking.

WHY TEAMS USE IT

- ✓ Less raw SQL to maintain.
- ✓ Parameterized queries by default.
- ✓ Models document app data shape.
- ✓ Migrations support schema changes.
- ✓ CRUD reads naturally in JavaScript.

TRADEOFF

- ✗ Can hide inefficient queries.
- ✗ Can make SQL feel optional.
- ✗ Association bugs can be confusing.
- ✗ Senior engineers still inspect generated SQL.

INSTALL AND CONNECT

Sequelize needs a PostgreSQL driver.

```
npm install sequelize pg pg-hstore

// db/index.js
const { Sequelize } = require("sequelize")

const db = new Sequelize(
  "postgres://localhost:5432/mydb",
  { logging: false }
)

module.exports = db
```

PACKAGES

- ✓ sequelize: the ORM.
- ✓ pg: PostgreSQL driver.
- ✓ pg-hstore: JSON serialization support.
- ✓ Connection string points at your database.

ONE MODEL, ONE TABLE

Models define database shape.

```
// models/User.js
const { DataTypes } = require("sequelize")
const db = require("../db")

const User = db.define("User", {
  username: { type: DataTypes.STRING, allowNull: false, unique: true },
  email: { type: DataTypes.STRING, allowNull: false, unique: true },
  bio: { type: DataTypes.TEXT }
})

module.exports = User
```

Sequelize adds id, createdAt, and updatedAt automatically.

TYPES BRIDGE

DataTypes map to PostgreSQL types.

STRING VARCHAR(255)	TEXT TEXT	INTEGER INTEGER
FLOAT FLOAT	BOOLEAN BOOLEAN	DATE / DATEONLY TIMESTAMP / DATE

DEVELOPMENT TABLE CREATION

db.sync() creates missing tables.

```
const db = require("./db")
require("./models/User")

db.sync()
  .then(() => {
    app.listen(8080)
  })
```

db.sync({ force: true }) drops and recreates tables. dev only.

CREATE AND READ

Every Sequelize query returns a Promise.

```
const user = await User.create({
  username: "ada_lovelace",
  email: "ada@example.com",
  bio: "First programmer"
})

console.log(user.id)
```

```
const users = await User.findAll()
const user = await User.findByPk(3)

const ada = await User.findOne({
  where: { username: "ada_lovelace" }
})
```

UPDATE AND DELETE

Find the row. Then change it.

```
// Pattern 1: instance update
const user = await User.findByPk(3)
if (!user) return res.sendStatus(404)

user.bio = "Updated bio"
await user.save()
```

```
// Delete instance
const user = await User.findByPk(3)
if (!user) return res.sendStatus(404)

await user.destroy()
res.sendStatus(204)
```

findByPk returns null when nothing exists. check before touching it.

RELATIONSHIPS

Associations teach models how to relate.

```
// models/index.js
const User = require("./User")
const Post = require("./Post")

User.hasMany(Post, { foreignKey: "userId" })
Post.belongsTo(User, { foreignKey: "userId" })

module.exports = { User, Post }
```

EAGER LOADING

include loads related data.

```
// User with posts
const user = await User.findPk(1, {
  include: Post
})

console.log(user.Posts)

// Posts with authors
const posts = await Post.findAll({
  include: User
})
```

WHY IT MATTERS

- ✓ Associated data is not loaded by default.
- ✓ include performs eager loading.
- ✓ Prevents many separate follow-up queries.
- ✓ This avoids the N+1 problem.

COMPLEX WHERE

Op turns SQL operators into JS.

```
const { Op } = require("sequelize")

await Product.findAll({
  where: { price: { [Op.gt]: 100 } }
})

await Product.findAll({
  where: { name: { [Op.like]: "%coffee%" } }
})
```

COMMON OPERATORS

- ✓ `Op.gt`, `Op.gte`, `Op.lt`, `Op.lte`
- ✓ `Op.ne` for not equal.
- ✓ `Op.like` for pattern matching.
- ✓ `Op.in` for lists.
- ✓ `Op.or` for OR conditions.

SEQUELIZE CHECKLIST

The ORM helps, but you still drive.

COMMON BUGS

- ✗ Forgetting `await`.
- ✗ Calling properties on `null`.
- ✗ Using `force: true` outside dev.
- ✗ Querying `include` before associations exist.
- ✗ Expecting `update` or `destroy` to return instances.

GOOD HABITS

- ✓ Require `models/index.js` at startup.
- ✓ Check generated SQL when confused.
- ✓ Use migrations on real projects.
- ✓ Use hooks for lifecycle behavior like password hashing.
- ✓ Drop to raw SQL when the ORM gets awkward.